# Data collection and annotation

We collected vehicle images from Kaggle and photographed parked cars ourselves. The aim was to build a smart parking prototype that reads a number plate, records an arrival, and calculates a charge when the same vehicle returns for exit.

## Preparing the labelled dataset

We used Roboflow to annotate number plates with bounding boxes and export labels in YOLO format. Each label stores a class ID and normalized box coordinates: centre x, centre y, width and height. These labels teach the detector where a plate is; they do not contain its text.

Before training, exact duplicate images were consolidated and annotations reconciled. Two visually confirmed missing boxes were added, and one image with no visible plate retained an empty label. Near-duplicates were not automatically checked. The original dataset and backup archive were preserved.

| Split | Images | Plate boxes |
|---|---|---|
| Training | 821 | 844 |
| Validation | 141 | 157 |
| Test | 199 | 274 |
| Total | 1,161 | 1,275 |

## Examples of the exported annotations




Figure 1. Sample dataset images with bounding boxes drawn from their saved YOLO labels. These are annotation examples, not model predictions.

The cleaned split counts are verified by the dataset audit. Exact source-wise counts and the Kaggle dataset URL were not recorded in that audit.

# Fine tuning YOLO11 on Colab

Because the local laptop did not have a GPU, we trained the detector in Google Colab using a Tesla T4. We uploaded and extracted the cleaned dataset, checked the train, validation and test folders, and configured their paths in data.yaml.

We initialized YOLO11n from pretrained weights and fine-tuned it for 30 epochs at an image size of 640 pixels with batch size 8. Training completed in 0.173 hours, approximately 10.4 minutes. The best checkpoint, best.pt, was downloaded for local CPU inference through FastAPI.

## Training evidence

```
      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
      30/30      1.61G     0.8778     0.4794      1.036          7        640: 100% ━━━━━━━━━━━━ 103/103 5.8it/s 17.8s
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 9/9 8.4it/s 1.1s
                   all        141        157      0.922      0.866      0.914      0.634

30 epochs completed in 0.173 hours.
Optimizer stripped from /content/parking_runs/plate_detector_yolo11n/weights/last.pt, 5.5MB
Optimizer stripped from /content/parking_runs/plate_detector_yolo11n/weights/best.pt, 5.5MB

Validating /content/parking_runs/plate_detector_yolo11n/weights/best.pt...
Ultralytics 8.4.176 🚀 Python-3.13.15 torch-2.11.0+cu130 CUDA:0 (Tesla T4, 14913MiB)
YOLO11n summary (fused): 101 layers, 2,597,051 parameters, 0 gradients, 6.4 GFLOPs
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 9/9 1.7it/s 5.3s
                   all        141        157      0.922      0.866      0.914      0.634
Speed: 1.4ms preprocess, 30.8ms inference, 0.0ms loss, 1.3ms postprocess per image
Results saved to /content/parking_runs/plate_detector_yolo11n
Best weights: /content/parking_runs/plate_detector_yolo11n/weights/best.pt
Run outputs: /content/parking_runs/plate_detector_yolo11n
```

Figure 2. Colab output showing the final epoch, completed training run and validation of best.pt. This evidence relates to detector training.

## Detector evaluation

| Split | Precision | Recall | mAP 50 | mAP 50–95 |
|---|---|---|---|---|
| Validation | 0.922 | 0.866 | 0.914 | 0.634 |
| Test | 0.938 | 0.763 | 0.824 | 0.575 |

Validation used 141 images and 157 plate instances; the held-out detector test used 199 images and 274 instances. High test precision indicates relatively few false detections, while lower recall shows that some plates were missed. These measures evaluate plate localization, not the accuracy of reading the characters.

After training, the local pipeline became: image or camera frame → YOLO plate box → cropped plate → OCR → parking session. Further detector tuning was not required for the OCR experiments described next.

# OCR experiments and improvement

We first used pretrained EasyOCR to read the YOLO crops. It correctly read several examples, but two cases exposed problems: a stacked plate prefix and white characters on a green background. We investigated preprocessing before selecting another engine.

## From EasyOCR to RapidOCR

For Cars320, the initial text MA2662KAJ09 had incorrect ordering and a misread hyphen. Resizing alone did not consistently resolve it. We allowed hyphens during recognition, resized crops toward 400 pixels, used beam decoding, and ordered text blocks spatially before removing separators. The result became KA09MA2662. We used the lowest block score so a weak prefix could not be hidden by a strong suffix.

EasyOCR still confused D with 0 on Cars281. We tested RapidOCR 3.10.0 with PP-OCRv6 models through ONNX Runtime 1.31.0 on the CPU. Its text detector initially missed the green crop; direct recognition of the whole plate succeeded. A fallback now requires original and inverted-grayscale readings to agree before accepting a wide, uncertain crop.

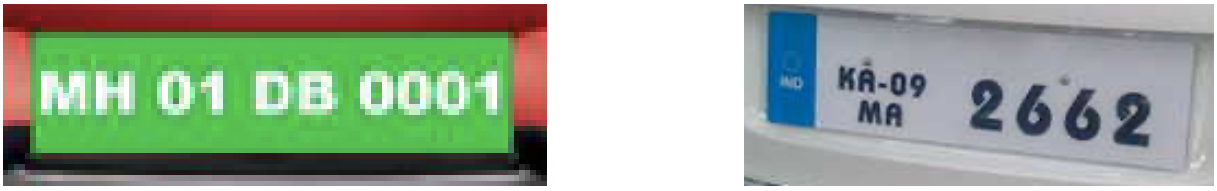



Figure 3. The green and stacked plates that motivated the OCR experiments, shown as enlarged crops from the original images.

| Image | Expected plate | EasyOCR | RapidOCR |
|---|---|---|---|
| Cars281 | MH01DB0001 | MH010B0001 | MH01DB0001 |
| Cars320 | KA09MA2662 | KA09MA2662 | KA09MA2662 |
| Cars307 | M906090K | M906090K | M906090K |
| Cars323 | TN99F2378 | TN99F2378 | TN99F2378 |

The improved EasyOCR pipeline matched 3 of 4 examples; RapidOCR with the fallback matched all 4. CPU OCR times were about 0.18–0.27 seconds for EasyOCR and 0.93–1.60 seconds for RapidOCR in this run. Timings excluded YOLO and could include lazy initialization. We selected RapidOCR despite the speed trade-off.

These previously inspected examples guided development; this is not a held-out OCR benchmark or proof of 100% accuracy. Confidence scores are not calibrated probabilities. Neither OCR engine was fine-tuned.

# Building the parking application

We connected the trained detector and OCR to a Python FastAPI backend and a browser dashboard built with HTML, CSS and JavaScript. SQLite stores parking sessions locally. The interface offers both image uploads and a live webcam preview, so the demonstration can use a phone displaying a plate image.

## Automatic entry and exit

Selecting an image starts recognition automatically. If the accepted plate has no active visit, the system records its arrival using server time. If that plate already has an active visit, the next accepted appearance closes it, calculates the charge and creates a receipt. Completed visits leave the active list but remain in history.

For the camera, three matching readings over at least two seconds are required. Holding a plate in view does not repeatedly toggle entry and exit. It must leave view until three blank readings span at least three seconds before the same plate can trigger again. Retried requests reuse event IDs to avoid duplicate database changes.

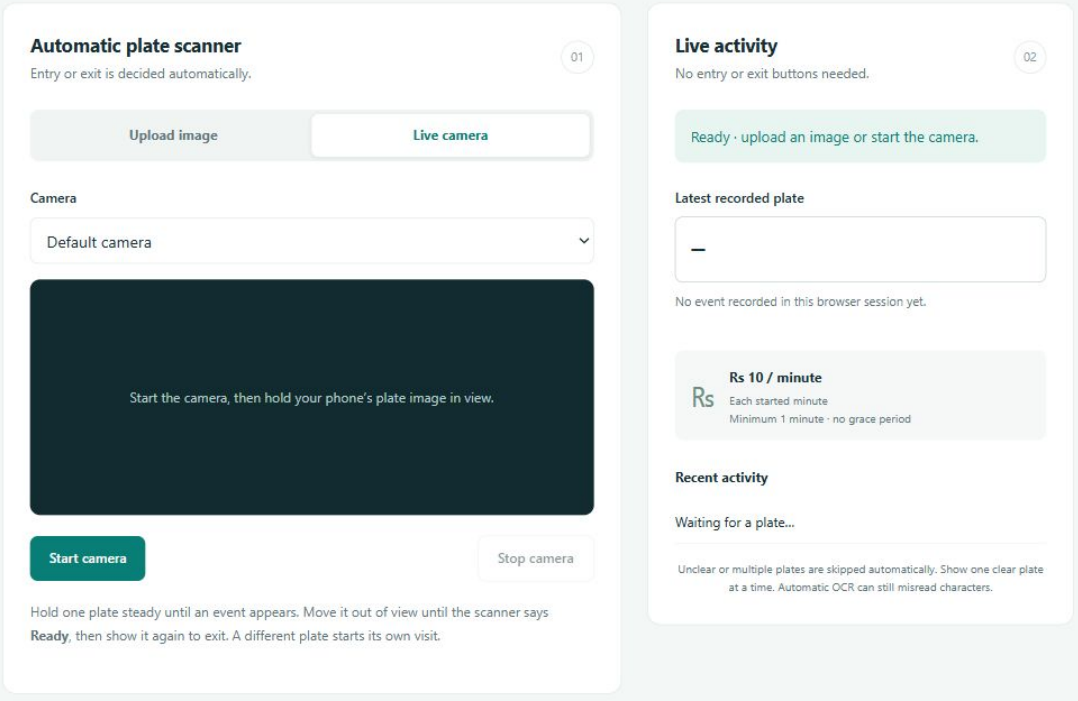



Figure 4. Dashboard evidence showing live-camera controls and automatic activity. This screenshot predates the later readable-background selection rule.

## Handling background vehicles

Initially, an image with two detected plates was rejected even if one was readable. We changed the rule: exactly one distinct plate passing detection ≥0.50, OCR ≥0.80 and format checks is processed, while unreadable background plates are ignored. Two distinct readable plates remain ambiguous. Cars311 verified that TN21AT8349 could enter despite an unreadable plate behind it.

# Billing verification and conclusion

The configured tariff is Rs 10 per started minute, with a minimum of one minute. An exit after 60 seconds costs Rs 10; after 61 seconds it costs Rs 20. The rate is saved at entry. Earlier visits keep their original tariff. Receipts show the plate, arrival, departure, duration and amount, and can be printed or saved as PDF.

## Verification of the complete workflow

Image tests verified recognition, entry, exit and receipt generation using a temporary database. A recorded example produced a Rs 10 receipt after a 24-second visit. The tests did not represent real paid parking activity.

The latest automated suite passed 24 tests covering billing boundaries, persistence, request retries, camera stability, OCR ordering, fallback agreement and readable-plate selection. A real-model test confirmed that Cars311 added its readable foreground plate once.

Camera-frame API tests checked repeat suppression and rearming. A systematic physical webcam and phone-screen accuracy evaluation remains to be completed. Payment collection is not implemented.

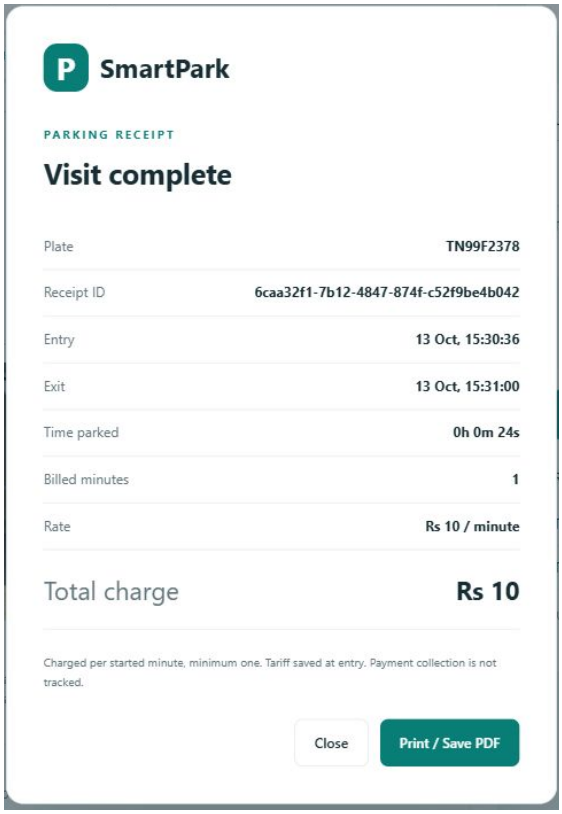



Figure 5. Saved test receipt at the minute tariff. History retains the completed visit.

## Conclusion and remaining work

The project progressed from mixed-source image collection and Roboflow labels to a Colab-trained detector and a working local parking application. OCR experiments improved difficult development examples and led to RapidOCR as the default. The prototype supports automatic visits and receipts on a CPU, but blur, glare, small text and competing readable plates can still prevent correct operation.

Next, a separate transcribed OCR set should measure full-plate accuracy, character errors and incorrect accepted readings. Live tests should measure entry–exit matching and total processing time across lighting and viewing conditions. These checks are needed before making broader accuracy claims.

**Evidence:** dataset_audit.txt; Colab training screenshot; ocr_comparison.json and compare_ocr.py; dashboard and receipt captures; automated tests. All are retained in the project. Collection and Roboflow workflow are described by the project author.